

Rajalakshmi Engineering College

Name: Mahathi B

Email: mahathi.b.2024.csd@rajalakshmi.edu.in

Roll no: 241701029

Phone: 9841411040

Branch: REC

Department: CSD

Batch: 2028

Degree: B.E - CSD

Scan to verify results



2024_28_IV_CSD_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Neha is working as a developer for CityMovie Theatre, which wants to build a system to calculate total ticket cost for movie-goers based on the number of tickets and type of seats booked.

Each customer's booking has:

Booking ID (integer) Customer Name (string) Number of Tickets (integer) Seat Type (string: "Standard", "Premium", "VIP")

The ticket prices are:

Standard – 250 units per ticket Premium – 400 units per ticket VIP – 600 units per ticket

The calculation rules:

Total Amount = Number of Tickets × Seat Price

If a customer books more than 4 tickets, they get a 10% discount on the total amount.

If the booking is for VIP seats and the total amount exceeds 3000 units, a 5% luxury tax is added after any discount.

Neha has been asked to implement this system using:

A class with attributes for booking details. A constructor to initialize booking details. Getter and Setter methods to retrieve and update booking details if required. A method to calculate the final ticket cost. Objects of the class to represent bookings.

Finally, display each customer's details and final ticket amount.

Input Format

The first line contains an integer N, representing the number of bookings.

For each booking:

- The next line contains the Booking ID (integer).
- The next line contains the Customer Name (string).
- The next line contains Number of Tickets (integer).
- The next line contains Seat Type ("Standard", "Premium", or "VIP").

Output Format

For each booking, print:

- Booking ID: <booking_id>
- Customer Name: <customer_name>
- Final Ticket Amount: <final_amount> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001
Ravi Kumar
3
Standard
Output: Booking ID: 1001
Customer Name: Ravi Kumar
Final Ticket Amount: 750.0

Answer

// You are using Java
import java.util.Scanner;

```
class Booking {  
    private int bookingId;  
    private String customerName;  
    private int numberOfTickets;  
    private String seatType;  
  
    // Constructor  
    public Booking(int bookingId, String customerName, int numberOfTickets,  
String seatType) {  
        this.bookingId = bookingId;  
        this.customerName = customerName;  
        this.numberOfTickets = numberOfTickets;  
        this.seatType = seatType;  
    }  
  
    // Getters  
    public int getBookingId() {  
        return bookingId;  
    }  
  
    public String getCustomerName() {  
        return customerName;  
    }  
  
    public int getNumberOfTickets() {  
        return numberOfTickets;  
    }  
  
    public String getSeatType() {  
        return seatType;  
    }  
}
```

```
}  
  
// Setters  
public void setBookingId(int bookingId) {  
    this.bookingId = bookingId;  
}  
  
public void setCustomerName(String customerName) {  
    this.customerName = customerName;  
}  
  
public void setNumberOfTickets(int numberOfTickets) {  
    this.numberOfTickets = numberOfTickets;  
}  
  
public void setSeatType(String seatType) {  
    this.seatType = seatType;  
}  
  
// Method to calculate final ticket amount  
public double calculateFinalAmount() {  
    double seatPrice = 0;  
  
    if (seatType.equals("Standard")) {  
        seatPrice = 250;  
    } else if (seatType.equals("Premium")) {  
        seatPrice = 400;  
    } else if (seatType.equals("VIP")) {  
        seatPrice = 600;  
    }  
  
    double totalAmount = numberOfTickets * seatPrice;  
  
    // Apply 10% discount if tickets > 4  
    if (numberOfTickets > 4) {  
        totalAmount = totalAmount * 0.9;  
    }  
  
    // Apply 5% luxury tax for VIP if total exceeds 3000 (after discount)  
    if (seatType.equals("VIP") && totalAmount > 3000) {  
        totalAmount = totalAmount * 1.05;  
    }  
}
```

```
        return totalAmount;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < N; i++) {
            int bookingId = sc.nextInt();
            sc.nextLine();

            String customerName = sc.nextLine();
            int numberOfTickets = sc.nextInt();
            sc.nextLine();

            String seatType = sc.nextLine();

            Booking booking = new Booking(bookingId, customerName,
            numberOfTickets, seatType);

            System.out.println("Booking ID: " + booking.getBookingId());
            System.out.println("Customer Name: " + booking.getCustomerName());
            System.out.printf("Final Ticket Amount: %.1f\n",
            booking.calculateFinalAmount());
        }

        sc.close();
    }
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Ravi is working as a developer for SecureLogin Systems, which wants to build a system to evaluate the strength of user passwords.

Each user record has:

User ID (integer) User Name (string) Password (string)

The system must calculate whether a password is strong or weak.

A password is considered strong if it meets all of the following conditions:

At least 8 characters long. Contains at least one uppercase letter. Contains at least one lowercase letter. Contains at least one digit. Contains at least one special character (from !@#\$%^&*).

Ravi has been asked to implement this system using:

A class with attributes for user details. A constructor to initialize user details. Getter and setter methods to retrieve or update user details. A method to check whether the password is strong. Objects of the class to represent users.

Finally, display each user's details and indicate whether their password is Strong or Weak.

Input Format

The first line contains an integer N, representing the number of users.

For each user:

The next line contains the User ID (integer).

The next line contains the User Name (string).

The next line contains the Password (string).

Output Format

For each user, print the details in the following format:

User ID: <user_id>

User Name: <user_name>

Password: <password>

Password Strength: <Strong/Weak>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

Abc@1234

Output: User ID: 1001

User Name: Ravi Kumar

Password: Abc@1234

Password Strength: Strong

Answer

// You are using Java

import java.util.Scanner;

class User {

private int userId;

private String userName;

private String password;

// Constructor

public User(int userId, String userName, String password) {

 this.userId = userId;

 this.userName = userName;

 this.password = password;

}

// Getters

public int getUserId() {

 return userId;

}

public String getUserName() {

 return userName;

}

```
public String getPassword() {  
    return password;  
}
```

```
// Setters
```

```
public void setUserId(int userId) {  
    this.userId = userId;  
}
```

```
public void setUsername(String userName) {  
    this.userName = userName;  
}
```

```
public void setPassword(String password) {  
    this.password = password;  
}
```

```
// Method to check password strength
```

```
public String checkPasswordStrength() {  
    if (password.length() < 8) {  
        return "Weak";  
    }  
}
```

```
boolean hasUpper = false;  
boolean hasLower = false;  
boolean hasDigit = false;  
boolean hasSpecial = false;
```

```
for (int i = 0; i < password.length(); i++) {  
    char ch = password.charAt(i);
```

```
    if (Character.isUpperCase(ch)) {  
        hasUpper = true;  
    } else if (Character.isLowerCase(ch)) {  
        hasLower = true;  
    } else if (Character.isDigit(ch)) {  
        hasDigit = true;  
    } else if ("!@#$%^&*".indexOf(ch) != -1) {  
        hasSpecial = true;  
    }  
}
```

```

        if (hasUpper && hasLower && hasDigit && hasSpecial) {
            return "Strong";
        } else {
            return "Weak";
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < N; i++) {
            int userId = sc.nextInt();
            sc.nextLine();

            String userName = sc.nextLine();
            String password = sc.nextLine();

            User user = new User(userId, userName, password);

            System.out.println("User ID: " + user.getUserId());
            System.out.println("User Name: " + user.getUserName());
            System.out.println("Password: " + user.getPassword());
            System.out.println("Password Strength: " +
            user.checkPasswordStrength());
        }

        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Anjali is working as a developer for CityFitness Gym, which wants to build

a system to calculate monthly membership fees for gym members based on the type of membership and the number of personal training sessions booked.

Each member's record has:

Member ID (integer) Member Name (string) Membership Type (string: "Basic", "Premium", "Elite") Number of Personal Training Sessions (integer)

The monthly fees are:

Basic – 1000 units Premium – 1500 units Elite – 2000 units

The cost of personal training sessions is 500 units per session.

The calculation rules:

Total Amount = Membership Fee + (Number of Personal Training Sessions × 500) If the number of sessions is more than 5, a 10% discount is applied on the total amount. If the member has Elite membership and the total amount exceeds 4000, an additional 5% service tax is added after discount.

Anjali has been asked to implement this system using:

A class with attributes for member details. A constructor to initialize member details. Getter and Setter methods to retrieve and update member details if required. A method to calculate the final monthly fee. Objects of the class to represent members.

Finally, display each member's details and the final monthly fee.

Input Format

The first line contains an integer N, representing the number of members.

For each member:

- Next line contains Member ID (integer)
- Next line contains Member Name (string)
- Next line contains Membership Type ("Basic", "Premium", "Elite")
- Next line contains Number of Personal Training Sessions (integer)

Output Format

For each member, print:

- Member ID: <member_id>
- Member Name: <member_name>
- Final Monthly Fee: <final_fee> (The final fee must be rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

Basic

3

Output: Member ID: 1001

Member Name: Ravi Kumar

Final Monthly Fee: 2500.0

Answer

```
import java.util.Scanner;
```

```
class Member {  
    private int memberId;  
    private String memberName;  
    private String membershipType;  
    private int sessions;
```

```
    // Constructor
```

```
    public Member(int memberId, String memberName, String membershipType,  
int sessions) {  
        this.memberId = memberId;  
        this.memberName = memberName;  
        this.membershipType = membershipType;  
        this.sessions = sessions;  
    }
```

```
    // Getters
```

```
    public int getMemberId() {
```

```
        return memberId;
    }
}
```

```
public String getMemberName() {
    return memberName;
}
```

```
public String getMembershipType() {
    return membershipType;
}
```

```
public int getSessions() {
    return sessions;
}
```

```
// Setters
```

```
public void setMemberId(int memberId) {
    this.memberId = memberId;
}
```

```
public void setMemberName(String memberName) {
    this.memberName = memberName;
}
```

```
public void setMembershipType(String membershipType) {
    this.membershipType = membershipType;
}
```

```
public void setSessions(int sessions) {
    this.sessions = sessions;
}
```

```
// Method to calculate final monthly fee
```

```
public double calculateFee() {
    double membershipFee = 0;
```

```
    if (membershipType.equals("Basic")) {
        membershipFee = 1000;
    } else if (membershipType.equals("Premium")) {
        membershipFee = 1500;
    } else if (membershipType.equals("Elite")) {
        membershipFee = 2000;
```

```
}  
double total = membershipFee + (sessions * 500);
```

```
// Apply 10% discount if sessions > 5  
if (sessions > 5) {  
    total = total - (total * 0.10);  
}
```

```
// If Elite and total > 4000, add 5% service tax AFTER discount  
if (membershipType.equals("Elite") && total > 4000) {  
    total = total + (total * 0.05);  
}  
return total;
```

```
}  
}  
  
public class Main {  
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);  
        int N = sc.nextInt();  
        sc.nextLine(); // consume newline
```

```
        for (int i = 0; i < N; i++) {
```

```
            int id = sc.nextInt();  
            sc.nextLine();
```

```
            String name = sc.nextLine();  
            String type = sc.nextLine();  
            int sessions = sc.nextInt();
```

```
            Member member = new Member(id, name, type, sessions);
```

```
            double finalFee = member.calculateFee();
```

```
            System.out.println("Member ID: " + member.getMemberId());  
            System.out.println("Member Name: " + member.getMemberName());  
            System.out.printf("Final Monthly Fee: %.1f\n", finalFee);  
        }
```

```
        sc.close();  
    }  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Each customer at the bank has an Account Number, Customer Name, and an Initial Balance. The bank allows two types of transactions:

Deposit – Increases the balance. Withdrawal – Decreases the balance, but only if enough funds are available. If the withdrawal amount exceeds the available balance, the transaction should be skipped, and the balance should remain unchanged.

You are required to implement this banking system by:

Creating a class with the necessary attributes to store account details.

Using a constructor to initialize the account details when a new account is created. Providing setter methods to update the details if required. Providing getter methods to retrieve account details. Creating objects of this class to represent different customers, where each customer can perform deposits and withdrawals.

Instructions:

Implement the class to store account details. Implement the logic for performing deposit and withdrawal transactions. Ensure that withdrawals don't exceed the available balance. After performing the transactions, print the account number, customer name, and final balance.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the account number (integer).

- The following line contains the customer name (string).
- The next line contains the initial balance (double).
- The next line contains the deposit amount (double).
- The next line contains the withdrawal amount (double).

Output Format

For each customer, print the details in the following format:

1. Account Number: <account_number>
2. Customer Name: <customer_name>
3. Final Balance: <final_balance> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1234

Rahul Sharma

5000

2000

3000

Output: Account Number: 1234

Customer Name: Rahul Sharma

Final Balance: 4000.0

Answer

```
// You are using Java
import java.util.Scanner;
```

```
class BankAccount {
    private int accountNumber;
    private String customerName;
    private double balance;
```

```
    // Constructor
    public BankAccount(int accountNumber, String customerName, double
balance) {
        this.accountNumber = accountNumber;
        this.customerName = customerName;
```

```
        this.balance = balance;
    }
}
```

```
// Getters
```

```
public int getAccountNumber() {
    return accountNumber;
}
```

```
public String getCustomerName() {
    return customerName;
}
```

```
public double getBalance() {
    return balance;
}
```

```
// Setters
```

```
public void setAccountNumber(int accountNumber) {
    this.accountNumber = accountNumber;
}
```

```
public void setCustomerName(String customerName) {
    this.customerName = customerName;
}
```

```
public void setBalance(double balance) {
    this.balance = balance;
}
```

```
// Deposit method
```

```
public void deposit(double amount) {
    balance += amount;
}
```

```
// Withdrawal method (only if sufficient balance)
```

```
public void withdraw(double amount) {
    if (amount <= balance) {
        balance -= amount;
    }
}
```

```
// else: ignore withdrawal
```

```
    }
}
```

```

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < N; i++) {

            int accNo = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();
            double initialBalance = sc.nextDouble();
            double depositAmount = sc.nextDouble();
            double withdrawalAmount = sc.nextDouble();

            BankAccount account = new BankAccount(accNo, name, initialBalance);

            account.deposit(depositAmount);
            account.withdraw(withdrawalAmount);

            System.out.println("Account Number: " + account.getAccountNumber());
            System.out.println("Customer Name: " + account.getCustomerName());
            System.out.printf("Final Balance: %.1f\n", account.getBalance());
        }

        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Neha is working as a developer for CityQuiz Platform, which wants to build a system to calculate quiz scores and identify top scorers among participants.

Each participant's record has:

Participant ID (integer) Participant Name (string) An array of scores in 5 quiz rounds (integers, each between 0 and 100)

The system must calculate:

Total Score = sum of scores in all 5 rounds. Average Score = Total Score ÷ 5. If a participant scores above 80 in all rounds, a bonus of 10 points is added to the total score. Identify the Top Scorer among all participants. If two participants have the same total score, the one with the lower Participant ID is considered the top scorer.

Neha has been asked to implement this system using:

A class with attributes for participant details. A constructor to initialize participant details. Getter and setter methods to retrieve or update participant details. A method to calculate total score and average score (including bonus if applicable). Objects of the class to represent participants.

Finally, display each participant's details and announce the Top Scorer.

Input Format

The first line of input contains an integer N, representing the number of participants.

For each participant:

- Next line: Participant ID (integer)
- Next line: Participant Name (string)
- Next line: 5 integers separated by spaces (scores for 5 quiz rounds)

Output Format

For each participant:

- Participant ID: <participant_id>
- Participant Name: <participant_name>
- Total Score: <total_score>
- Average Score: <average_score>

Finally, print "Top Scorer: <participant_name> with <total_score> points"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

85 90 88 92 87

Output: Participant ID: 1001

Participant Name: Ravi Kumar

Total Score: 452

Average Score: 90

Top Scorer: Ravi Kumar with 452 points

Answer

// You are using Java

```
import java.util.Scanner;
```

```
class Participant {  
    private int participantId;  
    private String participantName;  
    private int[] scores;
```

// Constructor

```
public Participant(int participantId, String participantName, int[] scores) {  
    this.participantId = participantId;  
    this.participantName = participantName;  
    this.scores = scores;  
}
```

// Getters

```
public int getParticipantId() {  
    return participantId;  
}
```

```
public String getParticipantName() {
    return participantName;
}

public int[] getScores() {
    return scores;
}

// Setters
public void setParticipantId(int participantId) {
    this.participantId = participantId;
}

public void setParticipantName(String participantName) {
    this.participantName = participantName;
}

public void setScores(int[] scores) {
    this.scores = scores;
}

// Method to calculate total score (with bonus if applicable)
public int calculateTotalScore() {
    int total = 0;
    boolean bonusEligible = true;

    for (int i = 0; i < 5; i++) {
        total += scores[i];
        if (scores[i] <= 80) {
            bonusEligible = false;
        }
    }

    if (bonusEligible) {
        total += 10;
    }

    return total;
}

// Method to calculate average score
public int calculateAverageScore() {
```

```

        return calculateTotalScore() / 5;
    }
}

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        sc.nextLine(); // consume newline

        Participant[] participants = new Participant[N];

        for (int i = 0; i < N; i++) {

            int id = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();

            int[] scores = new int[5];
            for (int j = 0; j < 5; j++) {
                scores[j] = sc.nextInt();
            }

            participants[i] = new Participant(id, name, scores);
        }

        Participant topScorer = participants[0];
        int topScore = topScorer.calculateTotalScore();

        for (int i = 0; i < N; i++) {

            int total = participants[i].calculateTotalScore();
            int average = total / 5;

            System.out.println("Participant ID: " + participants[i].getParticipantId());
            System.out.println("Participant Name: " +
participants[i].getParticipantName());
            System.out.println("Total Score: " + total);
            System.out.println("Average Score: " + average);
        }
    }
}

```

```
        if (total > topScore ||  
            (total == topScore &&  
             participants[i].getParticipantId() < topScorer.getParticipantId())) {  
  
            topScorer = participants[i];  
            topScore = total;  
        }  
    }
```

```
    System.out.println("Top Scorer: " + topScorer.getParticipantName() +  
        " with " + topScore + " points");
```

```
    sc.close();  
}
```

Status : Correct

Marks : 10/10